Module 02 – Bash Scripting (Revision Notes)

1. What is Bash?

- Bash stands for **Bourne Again Shell**.
- It is the default command-line shell in most Linux distributions.
- It allows users to interact with the Linux operating system by executing commands.

---

2. What is Bash Scripting?

- Bash scripting is the process of writing Linux commands in a file.
- The commands are executed automatically in sequence.
- It helps automate repetitive tasks and improves productivity.

---

3. Why Use Bash Scripts?

- Automates repetitive tasks.
- Saves time and effort.
- Reduces manual errors.
- Makes system administration easier.

---

4. Creating a Bash Script

Create a script using the Nano editor.

```
nano myfirstscript.sh
```

Save the file after writing the script.

---

5. Shebang (`#!/bin/bash`)

- The first line of a Bash script.
- Specifies that the script should be executed using the Bash shell.
- Ensures the correct interpreter is used.

Example:

```
#!/bin/bash
```

Module 02 – Bash Scripting (Revision Notes)

6. Comments

- Comments improve code readability.
- Ignored during script execution.
- Begin with the `#` symbol.

Example:

```
# This is a comment
```

---

7. Displaying Output (`echo`)

- `echo` is used to display text or variable values.
- Commonly used for messages and outputs.

Example:

```
echo "Hello World"
```

---

8. Variables

- Variables store values that can be reused.
- No spaces should be used around the `=` operator.

Example:

```
name="Misbah"
age=22
```

Access variables using `$`.

Example:

```
echo $name
```

---

9. User Input (`read`)

- The `read` command accepts input from the user.
- The entered value is stored in a variable.

Example:

Module 02 – Bash Scripting (Revision Notes)

```
read name
```

Display the entered value:

```
echo $name
```

---

10. Arrays

- Arrays store multiple values in a single variable.
- Each value is accessed using its index.

Example:

```
fruits=("Apple" "Mango" "Orange")
```

Access an element:

```
echo ${fruits[0]}
```

Modify an element:

```
fruits[1]="Banana"
```

---

11. Conditional Statements

Conditional statements allow scripts to make decisions.

if

Executes code when a condition is true.

elif

Checks another condition if the previous one is false.

else

Executes when none of the conditions are true.

Example:

```
if [ condition ]
then
    commands
```

Module 02 – Bash Scripting (Revision Notes)

```
elif [ condition ]
then
    commands
else
    commands
fi
```

---

12. Making a Script Executable

Grant execute permission.

```
chmod +x filename.sh
```

---

13. Running a Script

Execute the script.

```
./filename.sh
```

---

14. `ifconfig`

- Displays network interface information.
- Shows IP address and network configuration details.

---

15. Practical Exercise

Created a shell script to display a simple message.

Example:

```
#!/bin/bash

echo "The shell is fun."
```

Module 02 – Bash Scripting (Revision Notes)

Key Takeaways

- Understood the basics of Bash scripting.
- Learned to create and execute shell scripts.
- Practiced variables, user input, arrays, and conditional statements.
- Used Bash scripts to automate Linux commands.
- Gained practical experience with Linux scripting in Kali Linux.